

**People's Democratic Republic of Algeria**  
**Ministry of Higher Education and Scientific Research**



**Sciences faculty**  
**Département computer sciences**

## *Web Avancé et Microservices*

**Pipeline Microservice Complet Détection des Maladies des Plantes**

**présenté par : MOUSSAOUI DOUAA**

**BENNOUI YOUSRA**

**GOUFFI RYAD**

**Groupe : 1**

**Spécialité: MASTER 1 I2A**

**Année universitaire : 2025/2026**

## Table des Matières

|                                             |  |
|---------------------------------------------|--|
| 1. Introduction et Contexte.....            |  |
| 2. Architecture du Système.....             |  |
| 3. Microservices Implémentés.....           |  |
| 4. Service d'Authentification JWT.....      |  |
| 5. Communication Asynchrone — RabbitMQ..... |  |
| 6. Service Registry — Consul.....           |  |
| 7. Reverse Proxy — Traefik.....             |  |
| 8. Déploiement Docker.....                  |  |
| 9. Tests et Démonstration.....              |  |
| 10. Conclusion et Perspectives.....         |  |

# 1. Introduction et Contexte

Ce projet consiste à développer un pipeline microservices complet pour une application de détection des maladies des plantes. L'objectif est de concevoir une architecture distribuée respectant les principes des microservices, intégrant un modèle Deep Learning comme service indépendant.

## 1.1 Problématique

Les applications monolithiques présentent des limitations en termes de scalabilité, maintenance et déploiement. Une architecture microservices permet de décomposer l'application en services indépendants, chacun responsable d'une fonctionnalité spécifique.

## 1.2 Solution Proposée

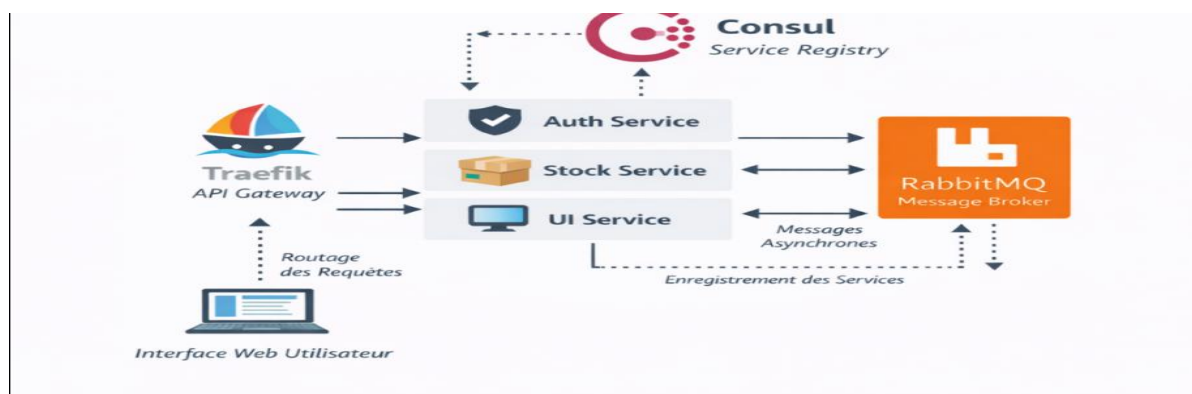
Nous proposons une architecture microservices complète comprenant : une application Django (UI + API REST), un microservice FastAPI (Deep Learning), RabbitMQ (messaging asynchrone), Consul (service registry) et Traefik (reverse proxy), orchestrés via Docker Compose.

## 1.3 Utilisateurs Cibles

| Utilisateur    | Groupe      | Fonctionnalités                                      |
|----------------|-------------|------------------------------------------------------|
| Agriculteur    | Agriculteur | Upload image, voir résultats personnels, historique  |
| Administrateur | Admin       | Dashboard global, statistiques, gestion utilisateurs |

# 2. Architecture du Système

## 2.1 Vue Globale de l'Architecture



## 2.2 Composants de l'Architecture

| Service          | Technologie         | Port       | Rôle                             |
|------------------|---------------------|------------|----------------------------------|
| UI + API REST    | Django + DRF        | 8000       | Interface web + endpoints CRUD   |
| Auth Service     | JWT (SimpleJWT)     | 8000       | Authentification et autorisation |
| Microservice DL  | FastAPI + PyTorch   | 5000       | Détection maladies par IA        |
| Message Broker   | RabbitMQ            | 5672/15672 | Communication asynchrone         |
| Service Registry | Consul              | 8500       | Découverte de services           |
| Reverse Proxy    | Traefik             | 80/8080    | Routage et load balancing        |
| Base de données  | SQLite → PostgreSQL | —          | Persistence des données          |

## 2.3 Flux Principal d'une Requête

| Étape | Description                                                   |
|-------|---------------------------------------------------------------|
| 1     | L'utilisateur accède via http://localhost (port 80)           |
| 2     | Traefik reçoit la requête et la route vers Django             |
| 3     | Django vérifie le token JWT via Auth Service                  |
| 4     | L'utilisateur upload une image de feuille                     |
| 5     | Django envoie l'image au microservice FastAPI (POST /predict) |
| 6     | FastAPI analyse l'image avec EfficientNet-B3                  |
| 7     | Le résultat JSON est retourné à Django                        |
| 8     | Django sauvegarde en base de données                          |
| 9     | Django publie un message dans RabbitMQ (async)                |
| 10    | L'utilisateur voit le résultat en temps réel                  |

## 3. Microservices Implémentés

### 3.1 Application REST Métier — Django

L'application Django constitue le cœur du système. Elle expose des endpoints CRUD et métiers via Django REST Framework, gère l'interface utilisateur et coordonne les communications entre les différents microservices.

| Méthode | Endpoint               | Description            | Auth          |
|---------|------------------------|------------------------|---------------|
| POST    | /api/auth/login/       | Obtenir token JWT      | Non           |
| POST    | /api/auth/refresh/     | Renouveler token       | Non           |
| GET     | /api/analyses/         | Liste des analyses     | JWT           |
| POST    | /api/analyses/predict/ | Analyser une image     | JWT           |
| GET     | /api/analyses/<id>/    | Détails d'une analyse  | JWT           |
| GET     | /                      | Interface upload image | Session       |
| GET     | /register/             | Créer un compte        | Non           |
| GET     | /admin-dashboard/      | Dashboard Admin        | Session+Admin |

### 3.2 Modèle de Données

Le modèle principal **PlantAnalysis** stocke les résultats de chaque analyse

```
class PlantAnalysis(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    image = models.ImageField(upload_to='plant_images/')
    plant_name = models.CharField(max_length=100)
    disease_name = models.CharField(max_length=100)
    confidence = models.FloatField()
    is_healthy = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

### 3.3 Microservice Deep Learning — FastAPI

Le microservice DL est un service indépendant développé avec FastAPI qui expose le modèle

EfficientNet-B3 via une API REST. Il reçoit une image, effectue la prédiction et retourne le résultat en JSON.

| Caractéristique    | Valeur                                  |
|--------------------|-----------------------------------------|
| Framework          | FastAPI (Python)                        |
| Modèle DL          | EfficientNet-B3 (PyTorch)               |
| Port               | 5000                                    |
| Endpoint principal | POST /predict                           |
| Format entrée      | image/jpeg, image/png (multipart)       |
| Format sortie      | JSON (plante, maladie, confiance, top3) |
| Temps de réponse   | < 2 secondes                            |

### 4.1 Présentation

L'authentification est gérée via **JSON Web Token (JWT)** en utilisant la bibliothèque django rest framework-simplejwt. Ce mécanisme est stateless — le serveur ne stocke pas l'état de session, chaque requête contient toutes les informations nécessaires.

### 4.2 Flux d'Authentification

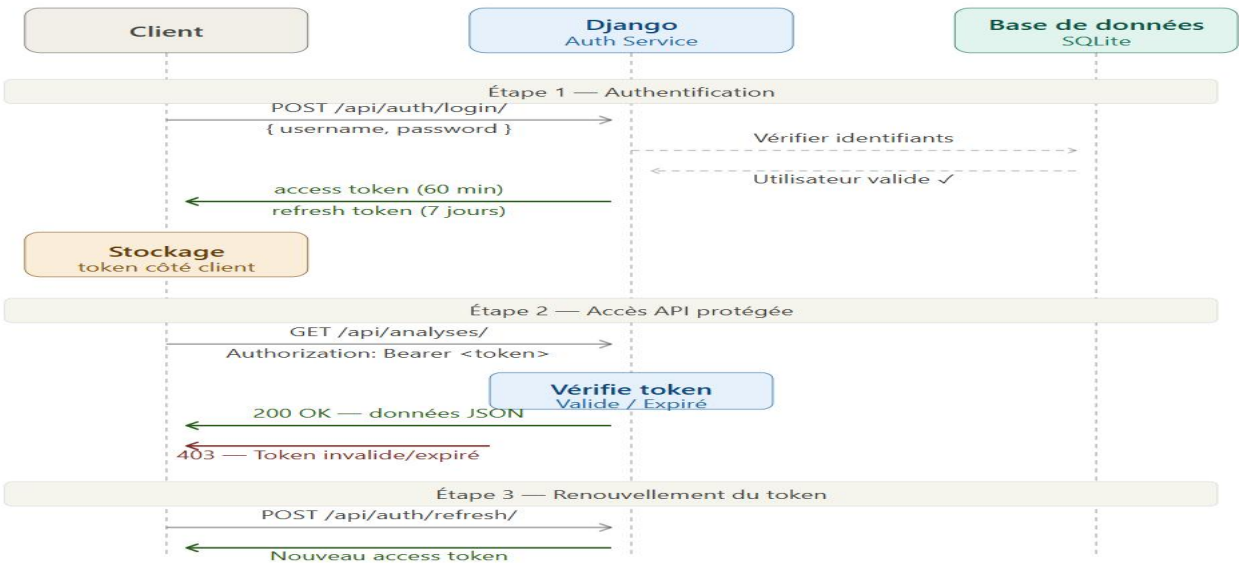


Figure 2 : Flux d'authentification JWT

## 4.3 Configuration JWT

```
# settings.py
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
}
```

## 4.4 Gestion des Groupes et Permissions

Deux groupes d'utilisateurs sont définis avec des permissions distinctes

| Groupe      | Création du compte           | Permissions                                                               |
|-------------|------------------------------|---------------------------------------------------------------------------|
| Agriculteur | Via /register/ (automatique) | Voir ses analyses uniquement<br>Upload d'images                           |
| Admin       | Via Django Admin (manuel)    | Voir toutes les analyses<br>Statistiques globales<br>Gestion utilisateurs |

## 4.5 Protection des Vues

```
# Vue protégée par JWT (API)
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def liste_analyses(request):
    analyses = PlantAnalysis.objects.filter(user=request.user)
    ...

# Vue protégée par Session (Web)
@login_required
def home(request):
    ...

# Vue Admin uniquement
@login_required
def admin_dashboard(request):
    if not request.user.is_superuser:
        return redirect('home')
    analyses = PlantAnalysis.objects.all() # Toutes les analyses
    ...
```

## 4.6 Screenshots de l'Interface

Figure 3 : Interfaces utilisateur

## 5.1 Présentation

RabbitMQ est un message broker qui permet la communication asynchrone entre les microservices. Dans notre architecture, après chaque analyse d'image, Django publie un message dans une file d'attente. Un worker consomme ces messages en arrière-plan sans bloquer l'interface utilisateur.

## 5.2 Architecture du Flux Asynchrone

| Étape | Composant | Action                                                  |
|-------|-----------|---------------------------------------------------------|
| 1     | Django    | Analyse terminée → publie message dans 'plant_analysis' |
| 2     | RabbitMQ  | Stocke le message dans la file d'attente (durable)      |
| 3     | Worker    | Consomme le message et effectue le traitement           |
| 4     | Worker    | Confirme la réception (basic_ack)                       |



## 5.3 Implémentation

```
# Publication asynchrone (non-bloquant)
def publier_async(analyse_id, user_id, image_path):
    thread = threading.Thread(
        target=publier_analyse,
        args=(analyse_id, user_id, image_path)
    )
    thread.daemon = True
    thread.start()

# Message publié dans RabbitMQ
message = json.dumps({
    'analyse_id': analyse_id,
    'user_id':     user_id,
    'image_path': image_path,
    'status':      'completed'
})
```

## 5.4 Dashboard RabbitMQ

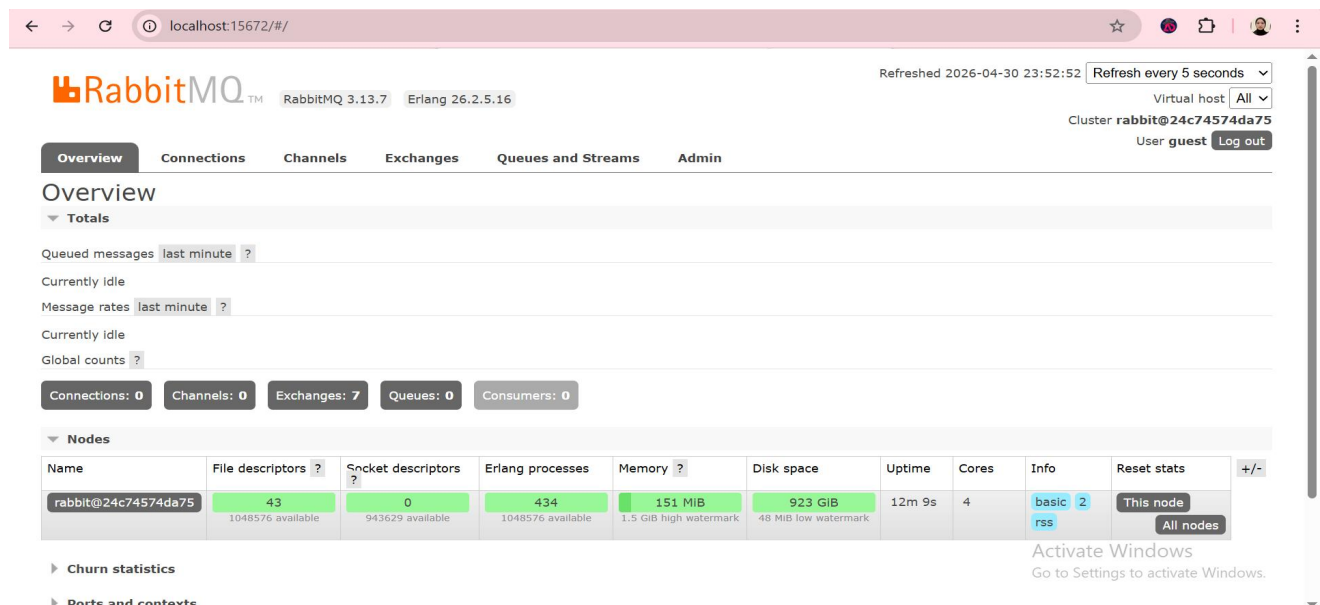


Figure 4 : Dashboard RabbitMQ — File plant\_analysis

## 6.1 Présentation

Consul est utilisé comme Service Registry pour enregistrer et découvrir les instances de microservices. Il joue le rôle d'annuaire — chaque service s'enregistre au démarrage et Consul maintient la liste des services disponibles avec leur état de santé.

## 6.2 Fonctionnalités Utilisées

- Enregistrement automatique des services au démarrage
- Health checks périodiques (toutes les 10 secondes)
- Découverte de services par nom (pas d'IP en dur)
- Suppression automatique des services défaillants
- Interface web sur <http://localhost:8500>

## 6.3 Dashboard Consul

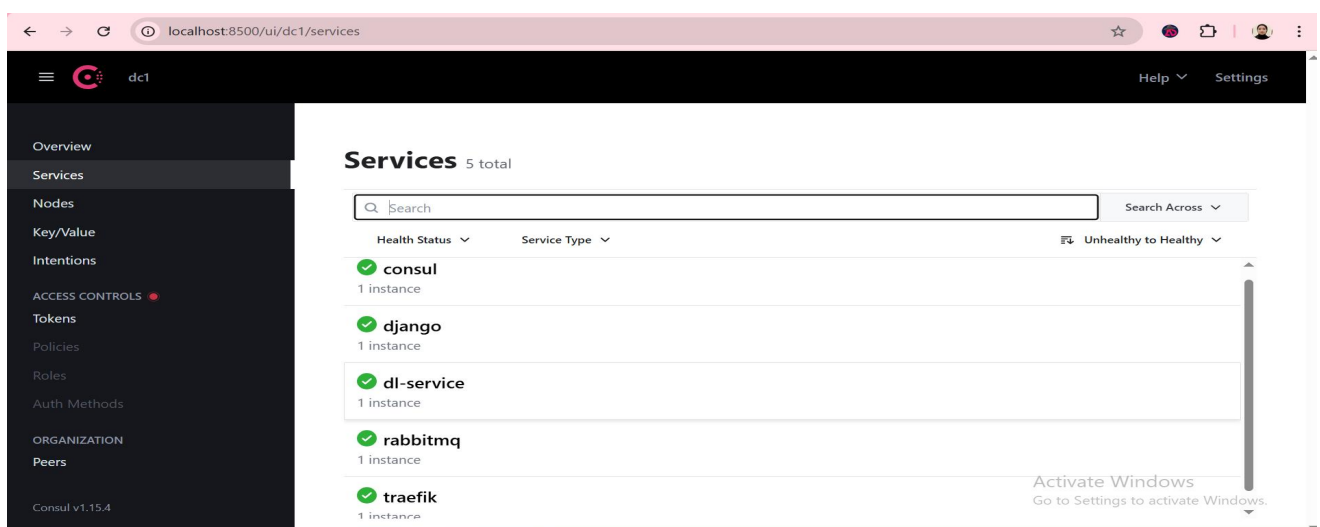


Figure 5 : Dashboard Consul — Services enregistré

## 7.1 Présentation

Traefik est placé en frontal de l'architecture comme reverse proxy et load balancer. Il reçoit toutes les requêtes entrantes sur le port 80 et les route dynamiquement vers le bon microservice en lisant les labels Docker.

## 7.2 Règles de Routage

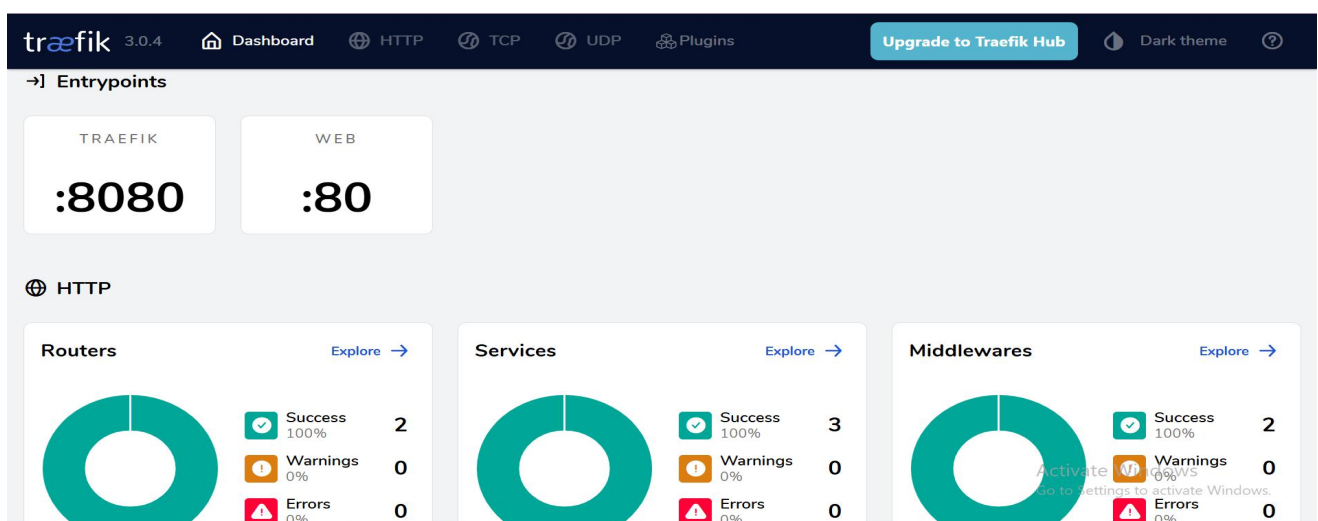
| URL       | Service cible        | Port |
|-----------|----------------------|------|
| /         | Django (UI + API)    | 8000 |
| /api/     | Django (API REST)    | 8000 |
| /predict/ | FastAPI (DL Service) | 5000 |
| /admin/   | Django (Admin Panel) | 8000 |

### 7.3 Configuration via Labels Docker

```
# Dans docker-compose.yml
django:
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.django.rule=PathPrefix(`/`)"
    - "traefik.http.services.django.loadbalancer.server.port=8000"

dl-service:
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.dl.rule=PathPrefix(`/predict`)"
    - "traefik.http.services.dl.loadbalancer.server.port=5000"
```

### 7.4 Dashboard Traefik



## 8. Déploiement Docker

### 8.1 Architecture Docker

L'ensemble des microservices est orchestré via Docker Compose. Chaque service tourne dans un container isolé sur un réseau Docker partagé (plant\_network).

### 8.2 Structure des Fichiers

```
plant_disease_project/  
■■■ Dockerfile           # Image Django  
■■■ docker-compose.yml   # Orchestration complète  
■■■ requirements.txt      # Dépendances Python  
■■■ manage.py  
■■■ plants/  
■■■ ...  
  
dl-service/  
■■■ Dockerfile           # Image FastAPI  
■■■ app.py  
■■■ plant_disease_model_finetuned.pth
```

### 8.3 docker-compose.yml

```
services:  
  traefik:  
    image: traefik:v3.0  
    ports: ["80:80", "8080:8080"]  
  
  consul:  
    image: consul:1.15  
    ports: ["8500:8500"]  
  
  rabbitmq:  
    image: rabbitmq:3-management  
    ports: ["5672:5672", "15672:15672"]  
  
  django:  
    build: .  
    ports: ["8000:8000"]  
    depends_on: [rabbitmq, consul]  
  
  dl-service:  
    build: ../dl-service  
    ports: ["5000:5000"]  
  
networks:  
  plant_network:  
    driver: bridge
```

## 8.4 Commande de Déploiement

```
# Lancer tous les services
docker-compose up --build

# Vérifier les containers actifs
```

```
docker ps

# Arrêter tous les services
docker-compose down
```

## 9. Tests et Démonstration

### 9.1 Tests Fonctionnels

| Fonctionnalité  | Test                         | Résultat                     |
|-----------------|------------------------------|------------------------------|
| Register        | Créer compte agriculteur1    | ✓ Groupe Agriculteur assigné |
| Login JWT       | POST /api/auth/login/        | ✓ Access + Refresh tokens    |
| Upload image    | Envoyer photo feuille tomate | ✓ Maladie détectée (98%)     |
| Admin Dashboard | Voir toutes les analyses     | ✓ Statistiques affichées     |
| RabbitMQ        | Message après analyse        | ✓ File plant_analysis active |
| API REST        | GET /api/analyses/ avec JWT  | ✓ Liste retournée            |
| Protection vues | Accès sans token             | ✓ Erreur 403                 |

### 9.2 Screenshots de l'Application



Figure 7 : Interface principale — Upload d'image

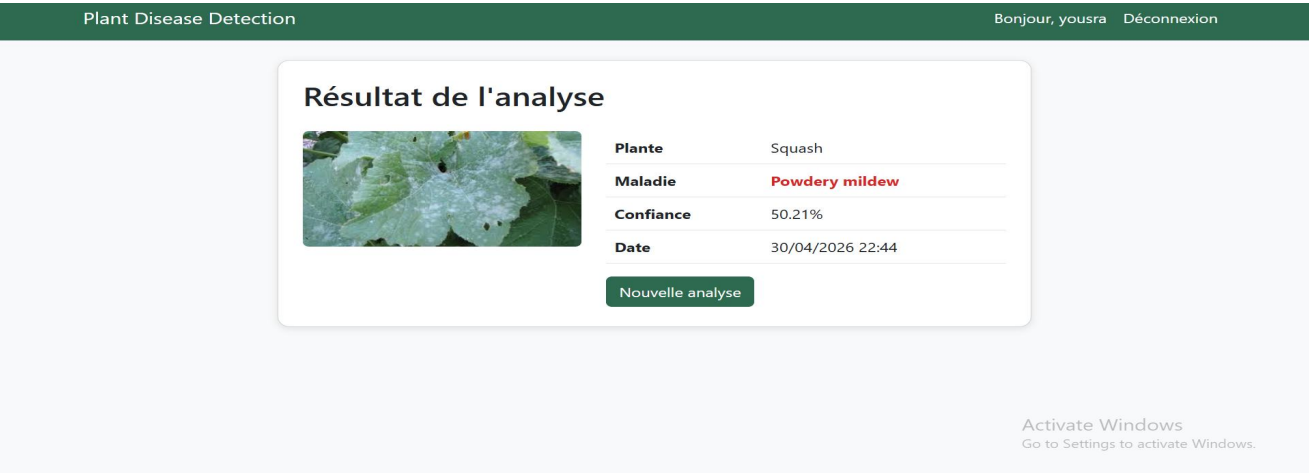


Figure 8 : Page de résultat — Maladie détectée avec confiance

# 10. Conclusion et Perspectives

## 10.1 Récapitulatif des Réalisations

| Contrainte WAMS               | Implémentation                     | Statut |
|-------------------------------|------------------------------------|--------|
| Application REST métier       | Django + DRF (endpoints CRUD)      | ✓      |
| Service d'authentification    | JWT (SimpleJWT) + Groupes          | ✓      |
| Application UI/UX             | Django Templates + Bootstrap 5     | ✓      |
| Communication asynchrone      | RabbitMQ (file plant_analysis)     | ✓      |
| Service Registry/Discovery    | Consul (port 8500)                 | ✓      |
| Reverse proxy / Load Balancer | Traefik (port 80)                  | ✓      |
| Déploiement multi-serveurs    | Docker Compose (5 containers)      | ✓      |
| Microservice DL               | FastAPI + EfficientNet-B0 (99.34%) | ✓      |

## 10.2 Conclusion

Nous avons développé avec succès un pipeline microservices complet pour la détection des maladies des plantes. L'architecture respecte toutes les contraintes techniques imposées par le cahier des charges WAMS. Le système intègre un modèle Deep Learning performant (99.28%) comme microservice indépendant, accessible via une interface web intuitive et sécurisée par JWT.

## 10.3 Perspectives d'Évolution

- Migration vers PostgreSQL pour la base de données en production
- Ajout de Kubernetes pour l'orchestration à grande échelle
- Mise en place d'un pipeline CI/CD (GitHub Actions)
- Monitoring avec Prometheus + Grafana
- Application mobile (React Native) pour les agriculteurs
- Internationalisation (Arabe, Français, Anglais)

